PROGRAMACIÓN II · UNIDAD 2 · CLASE 19

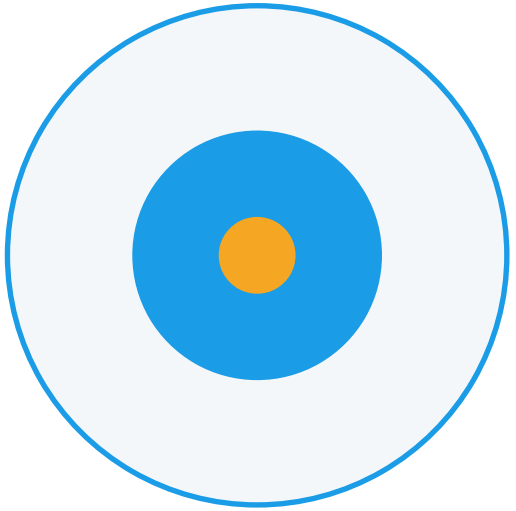
LSP, ISP y DIP

Los tres principios restantes de SOLID

> *Herencia correcta, interfaces enfocadas y dependencia de abstracciones.*

Objetivo de la clase

Cubrir los tres principios restantes, centrados en **herencia correcta**, **interfaces enfocadas** y **dependencia de abstracciones**.



Qué veremos hoy

L

LSP — Sustitución de Liskov

I

ISP — Segregación de interfaces

D

DIP — Inversión de dependencias

1 LSP — Sustitución de Liskov

Si B es una subclase de A, debemos poder reemplazar A por B sin que la aplicación altere su comportamiento.

- Pone de manifiesto el uso de clases polimórficas dentro de una jerarquía (Bárbara Liskov, 1987).
- Una subclase no debe romper lo que la base promete, por ejemplo lanzando excepciones donde la base no lo hace.

✓ **Dog reemplaza a Animal**

Hace ruido como se espera.

✗ **DumbDog no reemplaza a Dog**

Lanza una excepción: rompe el comportamiento.

Apunte Unidad 2, pág. 62-63

1 LSP — ejemplo: la jerarquía Animal

Animal.cs

```
public abstract class Animal
{
    public abstract void MakeNoise();
}

public class Dog : Animal
{
    public override void MakeNoise(){
        Console.WriteLine("bow wow");
    }
}

public class DumbDog : Dog      // ✗ rompe LSP
{
    public override void MakeNoise(){
        throw new Exception("I can't make noise");
    }
}
```

El problema

Un DumbDog no puede ocupar el lugar de un Dog: al llamar MakeNoise() lanza una excepción y la app deja de funcionar.

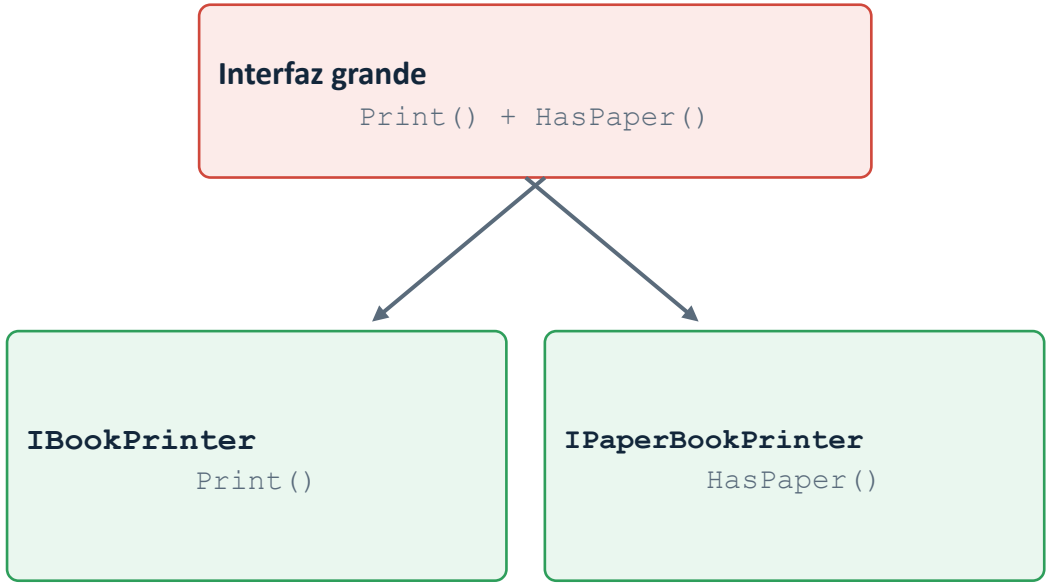
uso

```
Animal a = new DumbDog();
a.MakeNoise();    // excepción
```

Apunte Unidad 2, pág. 62-63

2 ISP — Segregación de interfaces

Las interfaces grandes deben dividirse en otras más pequeñas: ninguna clase debería verse forzada a implementar métodos que no usa.



- Cada clase implementa solo lo que le interesa.
- Varias interfaces chicas y enfocadas son mejores que una interfaz grande.

Apunte Unidad 2, pág. 63-64

2 ISP — ejemplo: impresión de libros

Printers.cs

```
public interface IBookPrinter
{
    void Print(string text);
}

public interface IPaperBookPrinter
{
    bool HasPaper();
}

// Imprime por consola: no necesita papel
public class BookPrinter : IBookPrinter { /* Print */ }

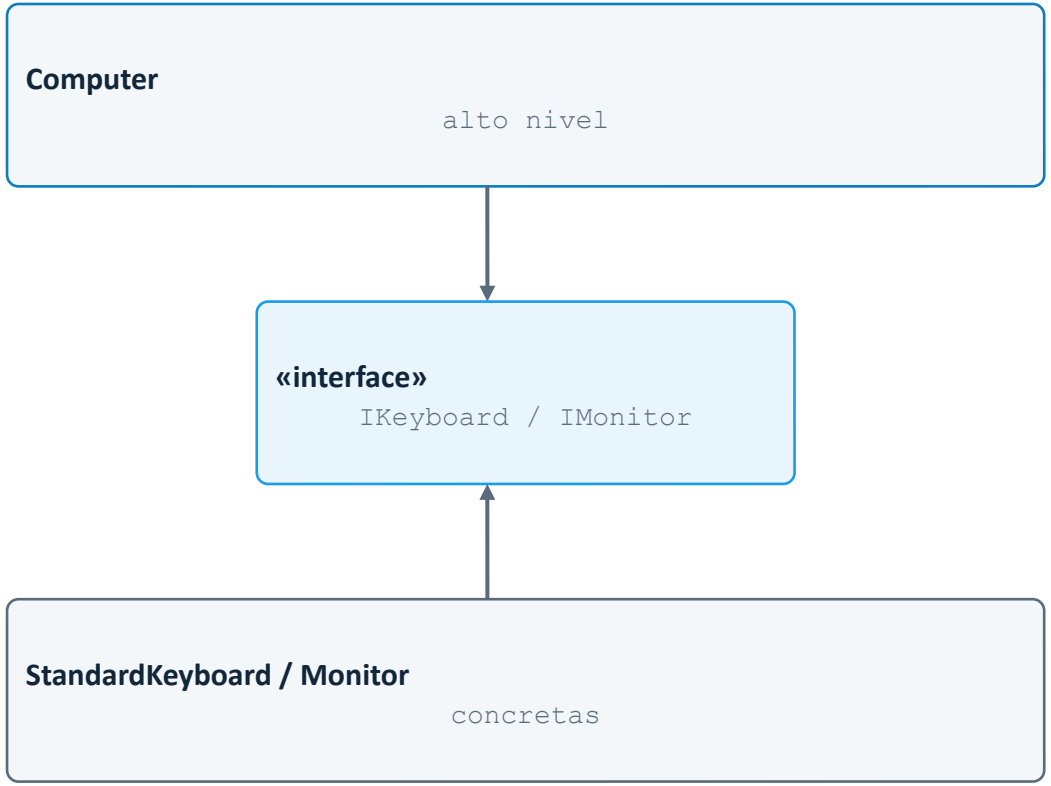
// Impresora de papel: implementa ambas
public class PaperBookPrinter
    : IBookPrinter, IPaperBookPrinter { /* Print + HasPaper */ }
```

- BookPrinter (consola) no está obligada a implementar HasPaper().
- Solo PaperBookPrinter, que sí usa papel, implementa IPaperBookPrinter.
- Cada clase depende únicamente de lo que necesita.

Apunte Unidad 2, pág. 63-64

3 DIP — Inversión de dependencias

Las clases deben depender de abstracciones (interfaces), no de clases concretas.



- Oculta la implementación real: si una clase cambia, la otra no se toca (desacoplamiento).
- Se logra con inyección de dependencias: se reciben desde afuera, por ejemplo por el constructor.

Apunte Unidad 2, pág. 64-66

3 DIP — ejemplo: la clase Computer

✗ **Antes: acoplado a clases concretas**

Computer (antes)

```
public class Computer
{
    private readonly StandardKeyboard keyboard;
    private readonly Monitor monitor;

    public Computer()
    {
        monitor  = new Monitor();
        keyboard = new StandardKeyboard();
    }
}
```

✓ **Después: depende de interfaces (DI)**

Computer (después)

```
public class Computer
{
    private readonly IKeyboard keyboard;
    private readonly IMonitor monitor;

    public Computer(IKeyboard keyboard,
                    IMonitor monitor)
    {
        this.keyboard = keyboard;
        this.monitor  = monitor;
    }
}
```

Ahora **el teclado y el monitor se pueden cambiar** por cualquier implementación de IKeyboard / IMonitor, sin modificar Computer.

Apunte Unidad 2, pág. 64-66

EN SÍNTESIS

- LSP: una subclase debe poder reemplazar a su base sin romper el comportamiento (DumbDog lo viola).
- ISP: interfaces chicas y enfocadas; nadie implementa métodos que no usa (IBookPrinter / IPaperBookPrinter).
- DIP: depender de abstracciones, no de clases concretas; inyectar las dependencias (Computer con IKeyboard / IMonitor).
- Los tres se apoyan en la herencia, las interfaces y el polimorfismo de la Unidad 1.

Próxima clase (práctica): corregir la violación de LSP (DumbDog), segregar interfaces e invertir dependencias en Computer.